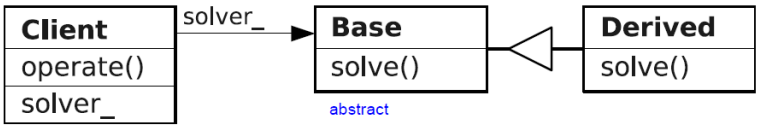
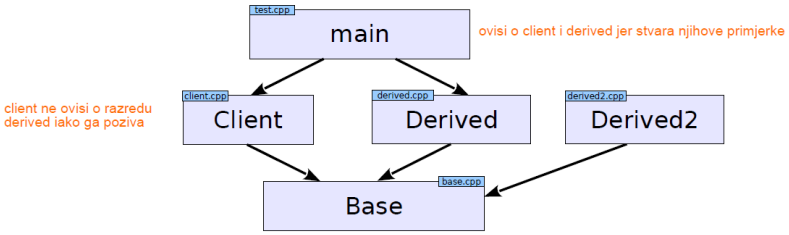
OOUP LAB1

Dinamički polimorfizam u C-u

Koncept OOP-a kojim se objekti različitih klasa ponašaju na različite načine kroz zajedničko sučelje i tako omogućava fleksibilnost i proširivost koda.



Client dio posla delegira metodi solve apstraktnog razreda Base. Derived nasljeđuje Base i definira njegovu virtualnu metodu solve. Client poznaje Base, ali ne poznaje Derived. Objekt tipa Client tako može preko Base pozivati metode razreda Derived o kojem ne ovisi. Značajno je zbog toga što možemo dodati još n razreda Derived_N s različitim implementacijama metode solve za različite objekte, a Client ih koristi jednako preko apstraktnog razreda Base. Također ne stvaramo domino efekt ovisnosti i ne povećavamo taj lanac do duljine n…Client ne poznaje sve dodane n razrede, samo razred Base. U C++ ovaj koncept možemo ostvariti korištenjem virtualnih metoda. Base definira čistu virtualnu metodu solve, a razred Derived koji nasljeđuje Base definira virtualnu metodu solve, njezinu funkcionalnost za objekte razreda Derived.



Po fizičkoj organizaciji vidimo da je dinamičkim polimorfizmom ovisnost usmjerena od složenog prema apstraktnom.

Kada u C++ pozovemo virtualnu funkciju, kompajler prozove tablicu virtualnih funkcija koji odg indexu, te prozove funkciju čiji je pointer tamo pohranjen. **Dinamički polimorfizam u C-u** možemo ostvariti korištenjem tablice pokazivača na funkcije. Njima ćemo imitirati virtualne metode (C++), pa ih nazivamo virtualnim tablicama. Omogućit ćemo dinamičko određivanje koje će se metode izvršiti u vrijeme izvođenja, na temelju tipa objekta nad kojim ih se pozove.

```
typedef char const* (*PTRFUN)();

PTRFUN dogVirtualTable[] = {dogGreet, dogMenu};
PTRFUN catVirtualTable[] = {catGreet, catMenu};
```

Definiramo tip podataka PTRFUN. To je pokazivač na funkciju koja ne prima nikakve argumente, a vraća const char*, tj. pokazivač na konstantni niz znakova. Inicijaliziramo dvije virtualne tablice za pse i mačke. Virtualne tablice u C-u su ostvarene kao polje pokazivača na funkcije, tako da im predajemo pokazivače na prethodno definirane funkcije.

```
struct Animal
{
    char *name;
    PTRFUN *functionsTable;
};
```

Struktura Animal predstvalja najopćenitiji razred (Base u prethodnom primjeru) dok su onda Dog i Cat objekti manje apstrakcije koji se naslanjanju na Animal (Derived razredi prethodnog primjera). Struktura sadrži pokazivač na

tablicu funkcija umjesto same tablice kako bi uskladili ponašanje objekata istog tipa te smanjili memorijski otisak polimorfnih objekata. Stvaranjem novih pasa ili mačaka, ne zauzimamo dodatni prostor za kopije istih tablica, nego svaki ima samo pokazivač na istu. Također, za stvaranje pasa i mačaka, memorija se alocira na način da ne ovisi o broju virualnih metoda pohranjenih u tablicama.

```
void animalPrintGreeting(struct Animal *animal) {
    printf("%s pozdravlja: %s\n", animal->name, animal->functionsTable[0]);
}

void animalPrintMenu(struct Animal *animal) {
    printf("%s voli %s\n", animal->name, animal->functionsTable[1]);
}
```

Za općeniti objekt Animal možemo definirati također općenite funkcije ispisa koje će pokazivačem dobivenog objekta životinje pristupiti virtualnoj tablici funkcija te životinje. Animal strukturom ostvarujemo općenitost, a prethodne funkcije definirali smo jednako za sve vrste životinja, dok se o specifičnosti ispisa brinu virtualne tablice.

```
void constructDog(Dog* this, const char* name) {
    constructAnimal(&(this->super), name);
    this->super.functionsTable=dogVirtualTable;
}

void constructCat(Cat* this, const char* name) {
    constructAnimal(&(this->super), name);
    this->super.functionsTable=catVirtualTable;
}
```

Funkcije kojima u prethodno alociranom memorijskom prostoru na kojeg pokazuje pokazivač na Animal strukturu, inicijaliziramo objekte psa ili mačke odabirom određene tablice funkcija.

```
struct Animal* createDog(const char* name){
    Dog* dog=(Dog*)malloc(sizeof(Dog));
    if (dog == NULL) {
        exit(EXIT_FAILURE);
    }
    constructDog(dog, name);
    return (Animal*)dog;
}

struct Animal* createCat(const char* name){
    Cat* cat=(Cat*)malloc(sizeof(Cat));
    if (cat == NULL) {
        exit(EXIT_FAILURE);
    }
    constructCat(cat, name);
    return (Animal*)cat;
}
```

Funkcije kojima kreiramo pse i mačke, zauzimamo memoriju potrebnu za strukturu, te inicijaliziramo objekte pozivom funkcije construct_().

Veza s terminologijom iz OO jezika:

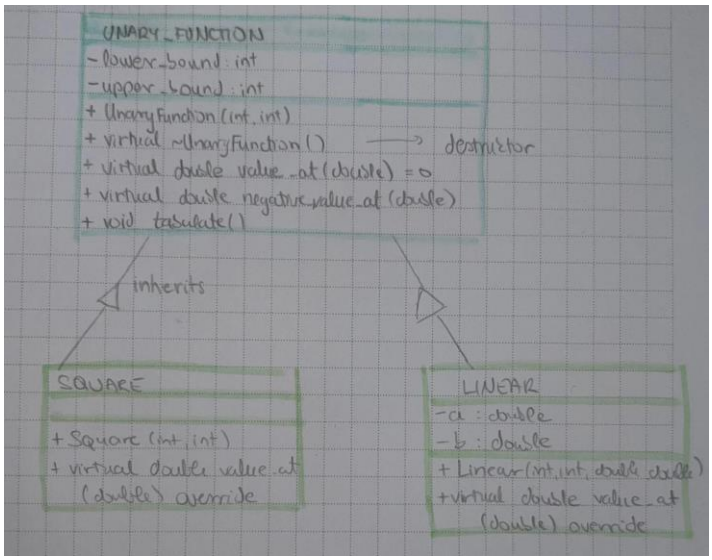
Podatkovni članovi objekta: npr. Animal predstavlja objekt, a name i functionsTable podatkovne članove koji sadrže informacije o objektu Animal. **Metode**: dogGreet, dogMenu, catGreet, catMenu. **Virtualne metode**: animalPrintGreet, animalPrintMenu -> zbog korištenja različitih tablica pokazivača na funkcije, njihovo ponašanje ovisi o tipu objekta koji primaju kao argument. **Konstruktori**: constructDog, constructCat -> inicijaliziraju objekte Dog/Cat. **Virtualne tablice**: dogVirtualTable, catVirtualTable s pokazivačima na funkcije koje definiraju ponašanje odgovarajućih tipova životinja.

Virtualne tablice u C++-u

Klasa Unary_Function- podatkovni članovi objekta: lower_bound, upper_bound (granice domene funkcije). Konstruktor: postavlja granice domene. Čisto virtualna funkcija value_at koja mora biti implementirana u izvedenim klasama. Virtualna funkcija negative_value_at s podrazumijevanom implementacijom koja se koristi ako se u izvedenim klasama ne definira nova implementacija. Metoda tabulate() ispisuje vrijednosti u domeni, statička funkcija same_functions_for_ints koja uspoređuje dvije funkcije po njihovim vrijednostima u određenom rasponu domene s određenom tolerancijom.

Klasa Square- nasljeđuje Unary_Function, u konstruktoru prosljeđuje granice roditeljskoj klasi, implementira virtualnu funkciju.

Klasa Linear- slično Square, ali ima dodatne podatkovne članove objekta, koeficijente a i b (ax + b).



Dijagram razreda

Tablice virtualnih funkcija razreda:

Unary_Function:

- virtual double value_at(double x) = 0; (null)
- virtual double negative_value_at(double x);

Square:

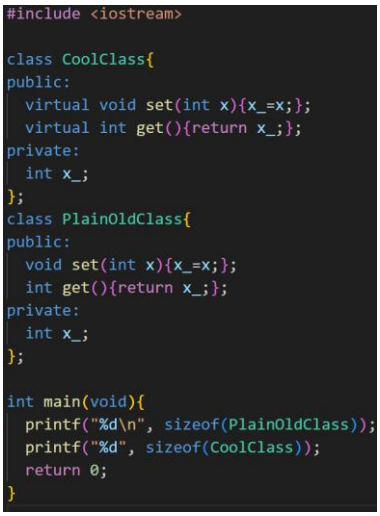
- virtal double value_at(double x); (impl)
- virtual double negative_value_at(double x);

Linear:

- virtual double value_at(double x); (impl)
- virtual double negative_value_at(double x);

negative_value_at se u Square i Linear uzima iz baznog razreda Unary_Function.

Memorijska cijena dinamičkog polimorfizma



```
#include <iostream>

class CoolClass{
public:
  virtual void set(int x){x_=x;};
  virtual int get(){return x_;};
private:
  int x_;
};
class PlainOldClass{
public:
  void set(int x){x_=x;};
  int get(){return x_;};
private:
  int x_;
};

int main(void){
  printf("%d\n", sizeof(PlainOldClass));
  printf("%d", sizeof(CoolClass));
  return 0;
}
```

PlainOldClass je klasa koja ne koristi virtualne funkcije, pa nema ni pokazivač na virtualnu tablicu. Veličina objekta PlainOldClass je 4 bajta zbog toga što objekt sadržava varijablu tipa int koji zauzima 4 bajta.
CoolClass objekt zauzima 16 bajtova

- 4 bajta za int podatak, kako i prije
- 8 bajtova za poiner na virtualnu tablicu s pokazivačima na virtualne funkcije koje koristi ova klasa (veličina pokazivača kod 64-bitne arhitekture je 8 bajtova)
- 4 bajta padding -> pristup podacima fiksne veličine riječi (64-bitna arhitektura: 8 bajtova) -> dodaje se padding do fiksne veličine granica arhitekture zbog poboljšanja performansi pristupa memoriji

Vremenska cijena dinamičkog polimorfizma

```
class Base{
public:
  //if in doubt, google "pure virtual"
  virtual void set(int x)=0;
  virtual int get()=0;
};
class CoolClass: public Base{
public:
  virtual void set(int x){x_=x;};
  virtual int get(){return x_;};
private:
  int x_;
};
int main(){
  PlainOldClass poc;
  Base* pb=new CoolClass;
  poc.set(42);
  pb->set(42);
}
```

Dijelovi assemblerskog koda u kojima se odvija alociranje memorije za objekte poc i *pb.

Na slici je prikazan dio koda kojim se alocira prostor na stogu za lokalne varijable, prije poziva funkcije main (call __main).

push rbp – rbp je 'base pointer' koji pokazuje na dno trenutnog okvira stoga, ta vrijednost se stavlja na vrh stoga kako bi se očuvala prije nego se promijeni unutar funkcije. Iduća naredba označava da se rbp stavlja na stog kao dio strukture obrade iznimki.

push rbx – jednako za registar rbx (za privremene podatke)

sub rsp, 56 – rezervacija 56 bajtova na stogu za okvir lokalnih varijabli i spremanje vrijednosti registara. rsp je također special purpose registar koji pokazuje na vrh trenutnog okvira stoga. Stog 'raste' smanjivanjem adresa, tako da oduzimamo vrijednost 56.

lea rbp, 48[rsp] – sada se postavlja bazni pokazivač stoga rbp relativnim adresiranjem, na adresu pomaknutu za 48 bajtova od početka trenutnog sadržaja pokazivača stoga; rsp. Bazni pokazivač rbp je sada na početku okvira funkcije main() koja će biti pozvana u nastavku. Na ovaj način je omogućeno jednostavno pristupanje lokalnim varijablama main funkcije relativnim adresiranjem u odnosu na rbp.

Call __main – poziv main funkcije – inicijalizacija prije pokretanja main() u programu

Dakle, u ovom isječku koda priprema se stog za poziv funkcije main, alocira se prostor za lokalne varijable koje će biti na stogu -> **objekt poc**.

```
main:
.LFB1843:
        push    rbp
        .seh_pushreg    rbp
        push    rbx
        .seh_pushreg    rbx
        sub     rsp, 56
        .seh_stackalloc 56
        lea     rbp, 48[rsp]
        .seh_setframe   rbp, 48
        .seh_endprologue
        call    __main
```

```
call    __main
mov     ecx, 16
call    operator new(unsigned long long)
mov     rbx, rax
mov     rcx, rbx
call    CoolClass::CoolClass()
```

Na drugoj slici je prikazan dio koda kojim se alocira prostor za *pb.

mov ecx, 16 – u ecx se sprema 16; veličina objekta koji se alocira

call operator new(..) – poziv funkcije new za alociranje memorije

mov rbx, rax – pohranjuje se povratna vrijednost funkcije; pokazivač na zauzetu memoriju, u rbx.

mov rcx, rbx – sada se ta vrijednost postavlja u rcx

call CoolClass::CoolClass() – pozivanje konstruktora za inicijalizaciju objekta CoolClass

*rbx, base registar opće namjene uobičajno korišten za indeksiranje kod pristupa podacima.

*rax, accumulator registar opće namjene koji se uobičajno koristi za rezultate aritmetičkih operacija ili za povratnu vrijednost funkcije, kao u ovom slučaju.

```
main:
.LFB1843:
    push    rbp
    .seh_pushreg    rbp
    push    rbx
    .seh_pushreg    rbx
    sub rsp, 56
    .seh_stackalloc 56
    lea rbp, 48[rsp]
    .seh_setframe   rbp, 48
    .seh_endprologue
    call    __main
    mov ecx, 16
    call    _Znwy
    mov rbx, rax
    mov rcx, rbx
    call    _ZN9CoolClassC1Ev
```

C++, Java omogućuju pisanje više funkcija istog imena i svaka uzima parametre različitih vrsta. Kako i se funkcije razlikovale, kodiraju se u asemblerski naziv niske razine koji jedinstveno identificira svaku verziju. (mangling) Na prethodnoj slici prikazan je taj kod bez prethodnog inverznog preslikavanja imena niske razine u imena korisničke razine.

Razlika u načinu alociranja objekata poc i *pb.

Za objekt poc memorija je alocirana na stogu jer je poc stvoren kao obična lokalna varijabla main metode. Memorija se alocira prije poziva main metode i sama se oslobađa završavanjem metode. Za objekt pb memorija je dinamički alocirana na gomili jer je korištena funkcija new. U ovom slučaju memorija se zauzima za vrijeme izvođenja programa i ostaje zauzeta do eksplicitne naredbe delete ili dok se program ne završi.

Dio assemblerskog koda koji je zadužen za poziv konstruktora objekta poc, ako postoji.

Za objekt poc ne postoji poziv konstruktora, ali postoji dio koda gdje se prvo računa adresa objekta poc na stogu, postavlja se vrijednost 42 u registar edx, a zatim se poziva metoda set koja postavlja vrijednost 42 registra edx na adresu iz registra rcx.

```
lea     rax, -12[rbp]
mov     edx, 42
mov     rcx, rax
call    PlainOldClass::set(int)
```

Dio koda zadužen za poziv konstruktora objekta *pb.

```
call    __main
mov     ecx, 16
call    operator new(unsigned long long)
mov     rbx, rax
mov     rcx, rbx
call    CoolClass::CoolClass()
mov     QWORD PTR -8[rbp], rbx
lea     rax, -12[rbp]
mov     edx, 42
mov     rcx, rax
call    PlainOldClass::set(int)
mov     rax, QWORD PTR -8[rbp]
mov     rax, QWORD PTR [rax]
mov     r8, QWORD PTR [rax]
mov     rax, QWORD PTR -8[rbp]
mov     edx, 42
mov     rcx, rax
call    r8
```

```
call    __main
mov ecx, 16
call    _Znwy
mov rbx, rax
mov rcx, rbx
call    _ZN9CoolClassC1Ev
mov QWORD PTR -8[rbp], rbx
lea rax, -12[rbp]
mov edx, 42
mov rcx, rax
call    _ZN13PlainOldClass3setEi
mov rax, QWORD PTR -8[rbp]
mov rax, QWORD PTR [rax]
mov r8, QWORD PTR [rax]
mov rax, QWORD PTR -8[rbp]
mov edx, 42
```

mov ecx, 16 – u ecx se sprema 16; veličina objekta koji se alocira

call operator new(..) – poziv funkcije new za dinamičko alociranje memorije na gomili

mov rbx, rax – pohranjuje se povratna vrijednost funkcije; pokazivač na zauzetu memoriju, u rbx.

mov rcx, rbx – sada se ta vrijednost postavlja u rcx (konstruktor u tom registru očekuje pokazivač na objekt)

call CoolClass::CoolClass() – pozivanje konstruktora za inicijalizaciju objekta CoolClass alociranog na gomili

Kako je prevoditelj izveo pozive pb->set i poc.set. Razlika? Koji poziv zahtijeva manje instrukcija? Za koju od te dvije izvedbe bi optimirajući prevoditelj mogao generirati kod bez instrukcije call, odnosno izravno umetnuti implementaciju funkcije – inlining?

```
mov     QWORD PTR -8[rbp], rbx
lea     rax, -12[rbp]
mov     edx, 42
mov     rcx, rax
call    PlainOldClass::set(int)
```

```
mov QWORD PTR -8[rbp], rbx
lea rax, -12[rbp]
mov edx, 42
mov rcx, rax
call    _ZN13PlainOldClass3setEi
```

poc. set

```
mov rax, QWORD PTR -8[rbp]
mov rax, QWORD PTR [rax]
mov r8, QWORD PTR [rax]
mov rax, QWORD PTR -8[rbp]
mov edx, 42
mov rcx, rax
call    r8
```

pb->set

poc.set – poziv se izvodi direktno, pozivom na objekt, statički jer je poc lokalni objekt. Prevoditelj ne mora koristiti pokazivače, vtable i sl., već može izravno umetnuti implementaciju funkcije umjesto poziva call (efikasnije izvršavanje) -> inlining. Ovaj poziv zahtijeva manje instrukcija jer ne pristupa virtualnoj tablici.

pb->set – izvodi se pomoću pokazivača na objekt na stogu kojim se pristupi vtable, a odgovarajućim indesom se iz tablice odabere konkretna implementacija virtualne metode. Ta metoda se poziva naredbom call. Ovdje inlining nije moguć jer se poziva dinamički, objekt može pokazivati na neku klasu koja je naslijedila base klasu koja može imati svoju posebnu implementaciju virtualne metode, što se otkriva pri izvođenju. Navedenim naredbama prvo se uzima vrijednost pokazivača lokalne varijable na stogu u registar rax. Čita se vrijednost memorije tog pokazivača i sprema se u rax (vtable). Vtable se učitava u registar r8, konstanta 42 u edx, a pokazivač lokalne varijable u rcx. Call r8 poziva metodu odgovarajućeg indeksa iz virtualne tablice. Dakle ostvaren je dinamički polimorfizam, određuje se koja se implementacija virtualne funkcije poziva ovisno o vrsti objekta.

*QWORD PTR: čitanje/pisanje 64-bitne vrijednosti memorije

Asemblerski kod za definiciju i inicijalizaciju tablice virtualnih funkcija razreda CoolClass.

```
        .globl  vtable for CoolClass
        .section        .rdata$vtable for CoolClass,"dr"
        .linkonce same_size
        .align 8
vtable for CoolClass:
        .quad   0
        .quad   typeinfo for CoolClass
        .quad   CoolClass::set(int)
        .quad   CoolClass::get()
```

Read-only sekcija koja sadrži tablicu virtualnih funkcija za razred CoolClass. U tablici se nalaze adrese virtualnih funkcija set i get razreda CoolClass.

```
CoolClass::CoolClass():
.LFB1849:
        push    rbp
        .seh_pushreg    rbp
        mov     rbp, rsp
        .seh_setframe   rbp, 0
        sub     rsp, 32
        .seh_stackalloc 32
        .seh_endprologue
        mov     QWORD PTR 16[rbp], rcx
        mov     rax, QWORD PTR 16[rbp]
        mov     rcx, rax
        call    Base::Base()
        lea     rdx, vtable for CoolClass[rip 16]
        mov     rax, QWORD PTR 16[rbp]
        mov     QWORD PTR [rax], rdx
        nop
        add     rsp, 32
        pop     rbp
        ret
```

```
Base::Base():
.LFB1846:
        push    rbp
        .seh_pushreg    rbp
        mov     rbp, rsp
        .seh_setframe   rbp, 0
        .seh_endprologue
        mov     QWORD PTR 16[rbp], rcx
        lea     rdx, vtable for Base[rip 16]
        mov     rax, QWORD PTR 16[rbp]
        mov     QWORD PTR [rax], rdx
        nop
        pop     rbp
        ret
        .seh_endproc
```

Unutar konstruktora CoolClass razreda postavlja se pokazivač na vtable razreda CoolClass. Taj pokazivač na tablicu nalazit će se na početku svakog objekta s virtualnim metodama. U ovih par naredbi vidljivo je da se poziva i konstruktor bazne klase. Na desnoj slici je prikaz naredbi koje se izvrše pozivom Base konstruktora gdje se lea naredbom (load effective address) učitava adresa vtable Base razreda u registar rdx , koja se pomoću njega onda postavi na određenoj adresi na stogu. Zatim, u CoolClass konstruktoru, adresa tablice virtualnih funkcija učitava se u rdx registar, da bi ju upisali u objekt s adresom na vrhu stoga. Tako je postavljen pokazivač na virtualnu tablicu na početak objekta i omogućen je pristup virtualnim funkcijama tablice.

Ručno prozivanje virtualne tablice pokazivačima na funkcije

*__cdedl: Specifikator konvencije poziva funkcije. Funkcija se poziva C konvencijom i sama je odgovorna za čišćenje stoga nakon izvršavanja. Konvencija je korištena za osiguravanje kompatibilnosti s načinom na koji su virtualne funkcije implementirane u C++-u; omogućen je ispravan prijenos pokazivača na matični objekt kojim pristupamo vtable. (https://www.geeksforgeeks.org/vtable-and-vptr-in-cpp/)

vPtr – virtual pointer, skriveni pokazivač kojeg dodaje prevoditelj kao člana razreda i pokazuje na vTable tog razreda.

```
uintptr_t* vTable = (uintptr_t*)(*(uintptr_t*)pb);

PTRFUN_1 f1 = (PTRFUN_1)vTable[0];
PTRFUN_2 f2 = (PTRFUN_2)vTable[1];
```

Prethodni izraz korišten je za dobavljanje adrese tablice virtualnih funkcija; vTable. uintptr_t je cjelobrojni tip podatka pogodan za pohranu adresa memorije jer garantira dovoljnu veličinu i može pohraniti cijelu adresu virtualne tablice. Eksplicitno označava da se radi o memorijskim adresama. (https://stackoverflow.com/questions/1845482/what-is-the-uintptr-t-data-type) Ako izraz gledamo s desna na lijevo, prvo pb (pokazivač na objekt klase B) pretvaramo u pokazivač na uintptr_t. Time je memorijska adresa objekta interpretirana kao niz uintptr_t vrijednosti koje indeksiramo kao niz i možemo pristupiti pojedinim dijelovima tablice. Dodatnim * dereferencira se pb i pristupi objektu na toj adresi -> na početku objekta je adresa virtalne tablice tog objekta (po prethodnom zadatku). Tu adresu također pretvaramo u pokazivač uintptr_t kako bi ju dalje koristili za pristup adresama virtualnih funkcija tablice. vTable je pokazivač na početak virtualne tablice koja je zapravo niz pokazivača na funkcije. vTable[1] pristupa drugom pokazivaču u tom nizu, tj. adresi druge virtualne funkcije.

Polimorfizam tijekom konstruiranja objekta

Različito ponašanje polimorfnih poziva tijekom i nakon završene konstrukcije objekta.

```
#include <stdio.h>

class Base{
public:
  Base() {
    metoda();
  }

  virtual void virtualnaMetoda() {
    printf("ja sam bazna implementacija!\n");
  }

  void metoda() {
    printf("Metoda kaze: ");
    virtualnaMetoda();
  }
};

class Derived: public Base{
public:
  Derived(): Base() {
    metoda();
  }
  virtual void virtualnaMetoda() {
    printf("ja sam izvedena implementacija!\n");
  }
};

int main(){
  Derived* pd=new Derived();
  pd->metoda();
}
```

```
Metoda kaze: ja sam bazna implementacija!
Metoda kaze: ja sam izvedena implementacija!
Metoda kaze: ja sam izvedena implementacija!
```

Base klasa ima virtualnu metodu koja se poziva iz nevirtualne metode, a ta nevirtualna metoda se poziva iz konsruktora klase Base. Derived klasa nasljeđuje Base i overridea virtualnu metodu svojom implementacijom. U main metodi stvara se jedan Derived objekt. New poziva konstruktor Derived, a on poziva konstruktor Base. U Base konstruktoru pozvana je nevirtualna metoda koja poziva virtualnu metodu, ali kako se konstruktor klase Derived još nije izvršio do kraja, vTable Derived klase još nije spremna, i uzima se implementacija virtualne metode baznog razreda, tj. izvodi se virtualna metoda iz razreda Base. Tada je završeno izvođenje konstruktora klase Base i nastavlja se izvođenje konstruktora Derived. Ponovno se poziva nevirtualna metoda baznog razreda. Konstruktor Derived je dovršen, te se iz vTable Derived razreda poziva izvedena implementacija virtualne metode. Nastavlja se main metoda u kojoj se ponovno poziva nevirtualna metoda iz same instance objekta. Tada se ponovno pristupa pokazivaču na vTable, te preko tablice dolazi do implementacije virtualne metode razreda Derived, te se ispisuje izvedeni ispis.

```
Derived::Derived():
.LFB51:
        push    rbp
        .seh_pushreg    rbp
        mov     rbp, rsp
        .seh_setframe   rbp, 0
        sub     rsp, 32
        .seh_stackalloc 32
        .seh_endprologue
        mov     QWORD PTR 16[rbp], rcx
        mov     rax, QWORD PTR 16[rbp]
        mov     rcx, rax
        call    Base::Base()
        lea     rdx, vtable for Derived[rip 16]
        mov     rax, QWORD PTR 16[rbp]
        mov     QWORD PTR [rax], rdx
        mov     rax, QWORD PTR 16[rbp]
        mov     rcx, rax
        call    Base::metoda()
        nop
        add     rsp, 32
        pop     rbp
        ret
```

```
Base::Base():
.LFB45:
        push    rbp
        .seh_pushreg    rbp
        mov     rbp, rsp
        .seh_setframe   rbp, 0
        sub     rsp, 32
        .seh_stackalloc 32
        .seh_endprologue
        mov     QWORD PTR 16[rbp], rcx
        lea     rdx, vtable for Base[rip 16]
        mov     rax, QWORD PTR 16[rbp]
        mov     QWORD PTR [rax], rdx
        mov     rax, QWORD PTR 16[rbp]
        mov     rcx, rax
        call    Base::metoda()
        nop
        add     rsp, 32
        pop     rbp
        ret
```

```
_ZN4BaseC2Ev:
.LFB45:
    push    rbp
    .seh_pushreg    rbp
    mov rbp, rsp
    .seh_setframe   rbp, 0
    sub rsp, 32
    .seh_stackalloc 32
    .seh_endprologue
    mov QWORD PTR 16[rbp], rcx
    lea rdx, _ZTV4Base[rip+16]
    mov rax, QWORD PTR 16[rbp]
    mov QWORD PTR [rax], rdx
    mov rax, QWORD PTR 16[rbp]
    mov rcx, rax
    call    _ZN4Base6metodaEv
    nop
    add rsp, 32
    pop rbp
    ret
    .seh_endproc
    .section .rdata,"dr"
```

U strojnom kodu prvo se u mainu poziva Derived konstruktor (call Derived::Derived()). On poziva Base konstruktor (1. slika) koji postavlja pokazivač na vTable (2. i 3. slika). 2. slika prikazuje demangled strojni kod 3. Slike. lea rdx, _ZTV4Base[rip+16] adresa vTable Base se stavlja u registar rdx, a zatim se sprema na memorijsku lokaciju na koju pokazuje pokazivač this (mem. lokacija 16[rbp])
This pokazivač pokazuje na trenutni objekt čija je metoda pozvana. C++ automatski proslijedi taj pokazivač na objekt kao skriveni parametar funkciji (zbog toga u prethodnim zadacima u C-u dodajem funkcijama this argument).
mov QWORD PTR [rax], rdxOva instrukcija adresu vTable iz rdx stavlja na mem. lokaciju na koju pokazuje rax, tj. na pokazivač this koji pokazuje na objekt. Zatim se u Base konstruktoru poziva Base::metoda().

```
call    printf(char const*, ...)
mov     rax, QWORD PTR 16[rbp]
mov     rax, QWORD PTR [rax]
mov     rdx, QWORD PTR [rax]
mov     rax, QWORD PTR 16[rbp]
mov     rcx, rax
call    rdx
```

Ovo je isječak instrukcija iz Base nevirtualne metode. This pokazivač na objekt metode se stavlja u rax registar (prethodno je tu na sami početak stavljen pokazivač na virtualnu tablicu). Zatim se 8-bajtni podatak (adresa) s adrese na koju pokazuje registar rax stavlja u registar rax. To znači da u rax registru imamo adresu prvog pokazivača virtualne funkcije iz vTable i sprema se u rdx, te je na kraju pozvana s call rdx instrukcijom. Poziva se nevirtualna metoda koja poziva virtualnu, ali iz Base razreda jer još traje konstruiranje Derived objekta.

Izvodi se Derived konstruktor koji postavlja novi pokazivač na vTable Derived razreda (prva slika). Naredni pozivi virtualne metode pokrenut će Derived implementaciju virtualne metode.